HELM PRODUCTION HANDBOOK

From Helm Fundamentals to Production Kubernetes Releases

Package Applications | Configure Easily | Deploy Reliably | Operate Safely | Upgrade & Rollback

WHAT YOU WILL MASTER

Helm Fundamentals | Chart Development | Templates | Values
Configuration | Install | Upgrade | Rollback | Hooks | Security
CI/CD Integration | GitOps | Troubleshooting | Production Best Practices

BUILD. DEPLOY. MANAGE. RECOVER.
SHIP APPLICATIONS WITH CONFIDENCE.

HELM EXPLAINED, WHY KUBERNETES NEEDS IT

1 THE PROBLEM HELM SOLVES

Kubernetes is powerful, but managing applications with raw YAML is hard.

- Many YAML files for one application
- Repeated across environments
- Easy to make mistakes
- Hard to update and maintain
- Not reusable

RAW KUBERNETES YAML AT SCALE

Deployment | Service | ConfigMap | Secret | Ingress | ...

Large, repetitive, error-prone, and difficult to reuse across environments.

2 WHAT A PACKAGE MANAGER MEANS

A package manager helps you install, upgrade, configure, and manage software consistently. It handles complexity so you can focus on delivering value.

HELM vs KUBECTL

HELM	KUBECTL
✓ Package manager for Kubernetes	✗ Kubernetes CLI
✓ Uses charts (reusable packages)	✗ Works with raw YAML
✓ Handles complex deployments	✗ One resource at a time
✓ Release management	✗ No release tracking
✓ Upgrades & rollbacks	✗ Manual rollbacks
✓ Environment configuration	✗ Harder env management
✓ Tracks history & state	✗ Harder to standardize

3 HELM IN A REAL DEVOPS WORKFLOW

Code Change → Build & Test Application → Push Image to Registry → Helm Chart + Values → Helm Install / Upgrade → Kubernetes Cluster

4 THE HELM MENTAL MODEL

CHART
The blueprint. Templates + Default values.

→

RELEASE
A running instance of a chart in a Kubernetes cluster.

→

KUBERNETES RESOURCES
The actual objects created in the cluster (Deployments, Services, Pods, etc.).

5 JUNIOR ENGINEER TIP

Helm does not replace Kubernetes knowledge.
You still need to understand Pods, Deployments, Services, Networking, Storage, and more.
Helm makes you faster, not less knowledgeable.

HELM ARCHITECTURE AND CORE COMPONENTS

HELM 3 ARCHITECTURE (NO TILLER)

1 HELM CLI

The command line tool engineers use to interact with Helm.
It packages charts, talks to the Kubernetes API, and manages releases.

2 KUBERNETES API

Helm communicates with the Kubernetes API server to create, update, or delete resources in the cluster.

3 CHARTS

A chart is a package of pre-configured Kubernetes manifests. It contains templates, defaults (values.yaml), and metadata to deploy an app.

4 RELEASES

A running instance of a chart in a Kubernetes cluster. Every install or upgrade creates a new revision in the release history.

5 REPOSITORIES

Places where charts are stored and shared. Helm can pull charts from public or private repositories.

6 RELEASE METADATA

Helm stores release metadata (revision history, status, values, hooks, timestamps) in the Kubernetes cluster using Secrets.

WHAT HAPPENED TO TILLER?

- In Helm 2, Tiller was a server-side component installed in the cluster.
- It handled all Helm operations.
- In Helm 3, Tiller was removed.
- Helm 3 is a client-side only architecture.
- This makes Helm 3 simpler, more secure, and easier to manage.

HOW HELM WORKS (HIGH LEVEL FLOW)

ENGINEER
Runs Helm commands

HELM (CLI)
Packages the chart, manages releases, and sends requests to Kubernetes API.

KUBERNETES API
Validates requests and stores release metadata.

KUBERNETES CLUSTER

Deployments, Services, ConfigMaps, Secrets, Pods, etc.

KEY TAKEAWAY

Helm acts as a package manager for Kubernetes. It takes a chart, applies your configuration, and reliably delivers it to the cluster while keeping track of every change.

VERIQTA

INSTALLING HELM AND YOUR FIRST COMMANDS

1 INSTALLING HELM

Helm 3 works on Linux, macOS, and Windows.
Example for Linux (using official install script):

```
curl -fsSL https://raw.githubeasoorcent.com/helm/helm/main/scripts/
get-helm-3 | bash
sudo mv helm /usr/local/bin/helm
helm version
```

TIP: Always install the latest stable version.
Check: https://helm.sh/docs/intro/install/

2 YOUR FIRST COMMANDS

helm version

Shows Helm client version.

```
$ helm version
version.BuildInfo{Version:"v3.14.2",
 GitCommit:"abc1234", GitTreeState:"clean",
 GoVersion:"go1.21.6"}
```

helm env

Shows Helm environment settings.

```
$ helm env
HELM_BIN="/usr/local/bin/helm"
HELM_HOME="/home/user/.config/helm"
HELM_CACHE_HOME="/home/user/.cache/helm"
HELM_PLUGINS="/home/user/.local/share/helm/plugins"
HELM_REGISTRY_CONFIG="/home/user/.config/helm/registry/config.json"
```

helm help

Shows help for Helm or any command.

```
$ helm help
The Kubernetes package manager

Common actions for Helm:
 install    Install a chart
 upgrade    Upgrade a release
 repo       Manage chart repositories
 search     Search for charts
 list       List releases
 ...
```

3 ADDING A REPOSITORY

helm repo add <name> <url>

Adds a new chart repository.

```
$ helm repo add bitnami https://charts.bitnami.com/bitnami
"bitnami" has been added to your repositories
```

helm repo update

Updates information about charts in all repositories.

```
$ helm repo update
Hang tight while we grab the latest from your chart
 repositories....
...Successfully got an update from the "bitnami" chart
 repository
 Update Complete.
```

helm search repo <keyword>

Searches for charts in repositories.

```
$ helm search repo nginx
NAME                    CHART VERSION   APP VERSION   DESCRIPTION
bitnami/nginx           15.9.0          1.25.4        NGINX Open Source...
bitnami/nginx-ingress   9.3.1           1.10.0        NGINX Ingress Controller...
bitnami/nginx-exporter  4.2.1           1.3.0         Prometheus exporter...
...
```

4 UNDERSTANDING COMMAND OUTPUT

PART	WHAT IT MEANS
NAME	The chart name (including the repo).
CHART VERSION	The chart version.
APP VERSION	The application version that the chart deploys.
DESCRIPTION	Short information about the chart.

TIP: Always read output carefully.
It tells you exactly what Helm is doing.

5 PRACTICE LAB

1. **Install Helm** — Install Helm on your machine and verify with helm version.
2. **Explore Helm** — Run helm env and helm help. Read the output and understand each line.
3. **Add a Repository** — Add the bitnami repository using helm repo add.
4. **Update Repositories** — Run helm repo update and confirm the update was successful.
5. **Search for a Chart** — Search for nginx using helm search repo nginx and read the results.

JUNIOR ENGINEER TIP: These commands are the foundation.
You will use them every day as a DevOps engineer.
Practice until they feel natural.

HELM

UNDERSTANDING A HELM CHART

A Helm chart is a package of pre-configured Kubernetes resources.
It contains everything Helm needs to install, configure, and manage an application.

WHAT A CHART CONTAINS

Chart.yaml
Metadata about the chart. Includes name, version, description, dependencies, and more.

values.yaml
Default configuration values for the chart. You can override these during install or upgrade.

templates/
Kubernetes manifest templates. Helm renders these files using values and creates real Kubernetes resources.

charts/
Holds dependent charts (subcharts). Used for chart dependencies.

templates/_helpers.tpl
Reusable template functions and definitions used across other templates.

.helmignore
Tells Helm which files and folders to ignore when packaging the chart.

CHART DIRECTORY DIAGRAM

```
my-chart/
├── Chart.yaml
├── values.yaml
├── charts/          # Subcharts or dependencies
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── _helpers.tpl
│   └── ...          # More templates
└── .helmignore
```

JUNIOR ENGINEER TIP

A well-structured chart is easy to read, customize, and maintain.
Keep templates simple and values flexible.

WHICH FILES HELM ACTUALLY RENDERS?

When you run a Helm command (install, upgrade, template), Helm renders only the files inside the templates/ directory (including _helpers.tpl if it is used). These templates are converted into Kubernetes YAML and sent to the Kubernetes API.

INPUT
values.yaml
+
Templates
(in templates/)

→ HELM →

RENDER ENGINE
Helm combines values with templates.

OUTPUT
Kubernetes YAML
(Rendered Manifests)

Sent to Kubernetes API and applied to the cluster.

REMEMBER
Helm packages, configures, and deploys. Kubernetes runs and manages the resources.
Learn Kubernetes deeply. Helm is a powerful helper, not a replacement.

BUILD YOUR FIRST HELM CHART

1 CREATE A NEW CHART

Helm has a command that scaffolds a ready-to-use chart structure.

```
$ helm create my-app
```

TIP: This creates a complete chart with common templates so you can start fast.

2 GENERATED DIRECTORY STRUCTURE

```
my-app/
├── Chart.yaml
├── values.yaml
├── charts/
├── templates/
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── serviceaccount.yaml
│   └── configmap.yaml
├── tests/
└── .helmignore
```

3 REMOVE UNNECESSARY TEMPLATES

We only need what our application actually uses.

COMMON FILES TO REMOVE

- ingress.yaml (if no Ingress)
- serviceaccount.yaml (if not needed)
- configmap.yaml (if not used)
- tests/ (optional)

Keep your chart simple. Only what you need.

4 CREATE A DEPLOYMENT

Edit templates/deployment.yaml to define how your app runs.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "my-app.fullname" . }}
  labels:
    {{- include "my-app.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "my-app.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "my-app.selectorLabels" . | nindent 8 }}
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: {{ .Values.image.repository }}:{{ .Values.image.tag }}
        ports:
        - containerPort: {{ .Values.service.port }}
```

5 CREATE A SERVICE

Edit templates/service.yaml to expose your application.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: {{ include "my-app.fullname" . }}
  labels:
    {{- include "my-app.labels" . | nindent 4 }}
spec:
  type: {{ .Values.service.type }}
  ports:
  - port: {{ .Values.service.port }}
    targetPort: {{ .Values.service.targetPort }}
    protocol: TCP
    name: http
  selector:
  {{- include "my-app.selectorLabels" . | nindent 4 }}
```

6 SET APPLICATION VALUES

Edit values.yaml to configure your app.

YAML

```yaml
replicaCount: 2
image:
  repository: nginx
  tag: "1.25-alpine"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
  targetPort: 80
```

7 LINT YOUR CHART

Always lint before installing or packaging.

```
$ helm lint my-app
==> Linting my-app
[INFO] Chart.yaml: icon is recommended
[INFO] templates/: all good
```

Linting catches syntax errors and common issues early.

8 FIRST REUSABLE APPLICATION CHART

You now have a clean, configurable, and reusable Helm chart.

- Easy to install
- Easy to configure
- Easy to upgrade
- Easy to share
- Production ready!

9 TRY IT (PRACTICE LAB)

1. **Create the chart**
   `$ helm create my-app`
2. **Edit values.yaml**
   Set image, replicas, and service type
3. **Lint the chart**
   `$ helm lint my-app`
4. **Install the chart**
   `$ helm install my-app ./my-app`
5. **Check resources**
   `$ helm list`
   `$ kubectl get all`

JUNIOR ENGINEER TIP: Start small, Understand every file. Helm becomes powerful when you can read, customize, and maintain your own charts.

HELM TEMPLATES WITHOUT THE CONFUSION

GO Ttemplating basics for Helm charts

{{ .Values... }}
Access values from values.yaml
Example: {{ .Values.image.tag }}
Gets the value of image.tag

{{ .Release... }}
Access release information
Example: {{ .Release.Name }}
Gets the name of the release

{{ .Chart... }}
Access chart information
Example: {{ .Chart.Name }}
Gets the name of the chart

VARIABLES

Set and reuse values in your templates.

```
{{- $name := .Values.name -}}
metadata:
  name: {{ $name }}
 labels:
   app: {{ $name }}
```

TIP: Variables start with $.
They help avoid repeating long values.

PIPELINES

Pass the output of one command to the next using | (pipe).

```
{{ .Values.image.repository |
   default "nginx" |
   quote }}
```

The value goes through default, then quote before being output.

TEMPLATE FUNCTIONS (COMMON)

default Use a default value if the given value is empty.

```
{{ .Values.image.tag | default "latest" }}
```

quote Wraps the value in double quotes.

```
{{ .Values.name | quote }}
```

required Fail the template if the value is empty.

```
{{ required "Value is required" .Values.name }}
```

HOW TEMPLATES BECOME KUBERNETES YAML

1. TEMPLATE FILE
You write a template with placeholders.

```
metadata:
  name: {{ .Values.name }}
  labels:
    app: {{ .Values.name }}
spec:
replicas: {{ .Values.replicaCount }}
template:
  spec:
    containers:
    - name: app
      image: {{ .Values.image.repository
          }}:{{ .Values.image.tag }}
```

→

2. VALUES PROVIDED
Values come from values.yaml or CLI.

```
name: my-app
replicaCount: 2
image:
 repository: nginx
 tag: 1.25
```

→

3. RENDER ENGINE
Helm replaces placeholders with actual values.

HELM

→

4. RENDERED YAML
Valid Kubernetes YAML is produced.

```
metadata:
  name: my-app
  labels:
    app: my-app
spec:
 replicas: 2
 template:
   spec:
     containers:
     - name: app
       image: nginx:1.25
```

→

5. APPLIED TO CLUSTER
The YAML is sent to the Kubernetes API.

KUBERNETES CLUSTER

COMMON TEMPLATE MISTAKES

1. WRONG INDENTATION
Bad indentation breaks YAML.

```
metadata:
  name: my-app    # WRONG
  labels:
    app: my-app
```

Always indent properly.
Use nindent when needed.

2. FORGETTING QUOTES
Strings with special chars should be quoted.

```
port: {{ .Values.port }}
# If port is "80", YAML
  may treat it as a number.
```

Use quote to force string type.

3. MISSING REQUIRED VALUES
Template renders but fails at runtime.

```
image: {{ .Values.image.repositorry }}
# If not set, you get
an empty image error.
```

Use required to fail early and clearly.

4. OVERUSING --SET
Hard to read and maintain.

```
helm install my-app ./my-app \
  --set image.tag=1.25 \
  --set replicaCount=2 \
  --set service.port=80
```

Prefer values files for complex configurations.

JUNIOR ENGINEER TIP

Templates are just text with placeholders. Helm replaces them with real values.
Keep templates simple, readable, and easy to maintain.

VERIQTA

MASTERING values.yaml

CONFIGURE ANY APPLICATION THE RIGHT WAY

1 DEFAULT CONFIGURATION

Every chart comes with a values.yaml that contains safe defaults.

```yaml
replicaCount: 2
image:
  repository: nginx
  tag: "1.25-alpine"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 512Mi
```

2 NESTED VALUES

Values can be deeply nested.

```yaml
database:
  enabled: true
  auth:
    username: admin
    password: secret
  primary:
    persistence:
      enabled: true
      size: 10Gi
```

3 LISTS AND MAPS

Helm supports lists and maps.

```yaml
tolerations:
  - key: "node-role.kubernetes.io/infra"
    operator: "Exists"
    effect: "NoSchedule"
env:
  - name: LOG_LEVEL
    value: "info"
  - name: FEATURE_X
    value: "true"
config:
  key1: value1
  key2: value2
```

4 ENVIRONMENT-SPECIFIC CONFIGURATION

Create separate values files for each environment.

values.yaml (base)	values-dev.yaml (development)	values-prod.yaml (production)
Common defaults	Dev overrides	Prod overrides

5 OVERRIDING VALUES FROM THE COMMAND LINE

`--set`	Override a value (auto type).	`helm install my-app ./my-chart \ --set replicaCount=3`
`--set-string`	Force a value to be a string.	`helm install my-app ./my-chart \ --set-string image.tag=1.25`
`--set-file`	Set a value from a file.	`helm install my-app ./my-chart \ --set-file config=./config.yaml`
`-f values-prod.yaml`	Use a values file.	`helm install my-app ./my-chart \ -f values-prod.yaml`

6 MULTIPLE VALUES FILES

You can pass multiple files. Later files override earlier ones.

```
helm install my-app ./my-chart -f values.yaml
-f values-dev.yaml -f values-prod.yaml
```

7 VALUES PRECEDENCE (LOWEST TO HIGHEST)

Chart values.yaml (defaults) → Additional values files (-f file.yaml) → --set-file (from file) → --set --set-string (CLI flags)

LOWEST PRIORITY → HIGHEST PRIORITY

8 WHY PRODUCTION TEAMS AVOID HUGE COMMAND-LINE OVERRIDES

HARD TO READ AND MAINTAIN	EASY TO BREAK	NOT REUSABLE	NOT COLLABORATIVE	HARD TO ROLLBACK	NOT SECURE
Long --set flags become unreadable and error-prone.	Typos or wrong paths can break deployments.	Configuration is not versioned or reusable.	Teams cannot review or understand CLI overrides easily.	You must remember exact flags to reproduce a release.	Sensitive data in CLI history or logs is a security risk.

JUNIOR ENGINEER TIP

Use values files for real environments.
Keep CLI overrides small and simple.

INSTALLING AND INSPECTING HELM RELEASES

DEPLOY, VIEW, AND UNDERSTAND YOUR APPLICATIONS

1 HELM INSTALL

Installs a chart as a new release.

```
helm install myapp bitnami/nginx
--namespace web
```

- myapp → Release name (must be unique in the namespace)
- bitnami/nginx → Chart (from a repo)
- --namespace web → Target namespace

WITH NAMESPACE CREATION

```
helm install myapp bitnami/nginx
--namespace web --create-namespace
```

Creates the namespace if it does not exist.

TIP

Choose clear release names. Example: payments-api, user-service, frontend-web

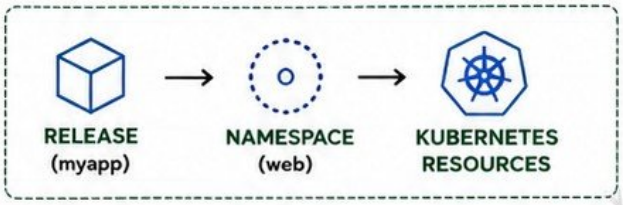
2 RELEASE NAMES

A release is an instance of a chart running in the cluster.

- Unique per namespace
- Identifies the deployment
- Used in Helm history and operations

3 NAMESPACES

- Helm releases live in a namespace.
- Helm will not create a namespace by default.
- Use --create-namespace to create it.



4 HELM LIST

```
helm list -A
```

Lists all releases.
-A shows all namespaces.

USE WHEN:

You want to see what releases are installed and where.

5 HELM STATUS

```
helm status myapp -n web
```

Shows detailed status of a release.

USE WHEN:

You want to check if the release is deployed, failed, or in progress.

6 HELM GET VALUES

```
helm get values myapp
-n web
```

Shows the values used for a release.

USE WHEN:

You want to review the configuration applied.

7 HELM GET MANIFEST

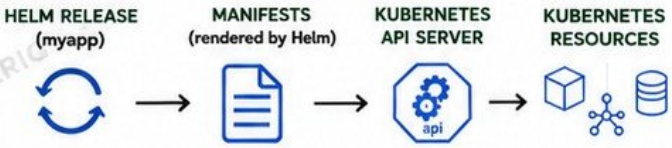
```
helm get manifest myapp
-n web
```

Shows the full Kubernetes manifests rendered by Helm.

USE WHEN:

You want to see exactly what Helm created in the cluster.

8 CONNECTING HELM STATE TO KUBERNETES RESOURCES



Helm renders the chart with your values, sends the manifests to the Kubernetes API, and Kubernetes creates the actual resources.

9 VERIFICATION WITH KUBECTL

```
kubectl get all -n web -l app.kubernetes.io/instance=myapp
```

Verifies the actual resources created by the release.

Other useful checks:

```
kubectl get pods -n web
kubectl describe deployment -n web
kubectl get svc -n web
kubectl get events -n web --sort-by=.metadata.creationTimestamp
```

10 QUICK WORKFLOW SUMMARY

JUNIOR ENGINEER TIP

Always verify with kubectl. Helm tells you what it asked Kubernetes to do. Kubernetes shows you what actually happened.

RENDERING AND VALIDATING BEFORE DEPLOYMENT

CATCH PROBLEMS EARLY. DEPLOY WITH CONFIDENCE.

1 WHY ENGINEERS INSPECT MANIFESTS FIRST

- Templates can generate incorrect YAML.
- Small mistakes can break deployments.
- Catch issues before they reach the cluster.
- Save time, avoid failed rollouts, and reduce risk in production.

JUNIOR ENGINEER TIP

Never deploy blindly. Always render, inspect, and validate before applying changes to a cluster.

2 HELM COMMANDS FOR RENDERING & VALIDATION

COMMAND	WHAT IT DOES	WHEN TO USE IT
`helm template myapp ./mychart -f values.yaml`	Renders templates locally and outputs Kubernetes YAML to the terminal.	Always first. Inspect the generated manifests.
`helm lint ./mychart`	Checks chart for common problems and best practices.	Before committing or deploying.
`helm install myapp ./mychart -f values.yaml --dry-run --debug`	Simulates an install. Renders and validates without sending to the cluster.	Final check before real installation.

3 TEMPLATE RENDERING FAILURES

- Syntax errors in templates
- Missing required values
- Using values that don't exist
- Wrong data type (string vs number)
- Invalid functions or pipelines

EXAMPLE ERROR:

```
Error: template: mychart/templates/deploy.yaml:15:18:
executing "mychart/templates/deploy.yaml" at <.Values.image.tag>:
map has no entry for key "tag"
```

HOW TO FIND & FIX

- Run: `helm template ... --debug`
- Read the error line and file
- Fix the template or supply the missing value

4 YAML VALIDATION

- YAML must be syntactically correct.
- Wrong indentation, colons, or characters will break rendering.

CHECK WITH:

```
helm template myapp ./mychart |
yamllint -
```

COMMON YAML ISSUES

- Wrong indentation
- Missing colon (:)
- Tabs instead of spaces
- Unclosed quotes

5 KUBERNETES SCHEMA VALIDATION

Even if YAML is correct, the manifest may still be invalid for Kubernetes.

CHECK WITH:

```
helm template myapp ./mychart |
kubeconform -strict -ignore-missing-schemas
```

- Validates resource types
- Validates fields and structure
- Catches unsupported or removed APIs

PRODUCTION AWARENESS

Invalid manifests can cause failed deploys, partial deployments, or downtime. Validate before you deploy.

6 PRE-DEPLOYMENT WORKFLOW (THE SAFE PATH)

1. RENDER

`helm template`
Render all templates locally to view the generated YAML.

→

2. INSPECT

Read through the manifests. Check resources, values, labels, and configurations.

→

3. VALIDATE

Run `helm lint`, YAML validation, and Kubernetes schema validation.

→

4. DEPLOY

`helm install` (or upgrade)
Deploy to the cluster only after validation passes.

→

5. VERIFY

Use `kubectl` to confirm resources are created and the app is healthy.

REMEMBER THIS

Render → Inspect → Validate → Deploy → Verify
This simple habit prevents most production issues.

WHEN TO STOP AND FIX

- If rendering fails → fix templates or values.
- If validation fails → fix YAML or schema issues.
- If something looks wrong → don't deploy.

Fix early. Save later.

LABELS, NAMES, HILPERS, AND REUSABLE TEMPLATES

WRITE ONCE. REUSE EVERYWHERE. KEEP CHARTS MAINTAINABLE.

1 KUBERNETES RECOMMENDED LABELS

Use standard labels so tools and humans can understand your resources.

LABEL	PURPOSE
app.kubernetes.io/name	Name of the application
app.kubernetes.io/instance	Helm release name
app.kubernetes.io/version	Application version
app.kubernetes.io/managed-by	Tool managing the app (Helm)
app.kubernetes.io/component	Component name (api, web, db)
app.kubernetes.io/part-of	Part of a larger application
app.kubernetes.io/created-by	Created by Helm

2 NAMING CONSISTENCY

Consistent names = easier to manage, filter, and automate.

- Use release name for unique resources.
- Use helper templates for all names.
- Keep names DNS-1123 compliant.
- Avoid hardcoding names in templates.
- Use the same naming across all charts in your organization.

3 _HELPERS.TPL – THE HEART OF MAINTAINABLE CHARTS

Store reusable template logic in this file and include it anywhere.

```
mychart/
└─ templates/
   ├─ _helpers.tpl
   ├─ deployment.yaml
   ├─ service.yaml
   └─ configmap.yaml
values.yaml
Chart.yaml
```

```
{{/* mychart/templates/_helpers.tpl */}}
{{- define "mychart.name" -}}
{{- default .Chart.Name .Values.nameOverride
   | trunc 63 | trimSuffix "-" -}}
{{- end -}}

{{- define "mychart.fullname" -}}
{{- if .Values.fullnameOverride -}}
{{- .Values.fullnameOverride | trunc 63 | trimSuffix "-" -}}
{{- else -}}
{{- printf "%s-%s" .Release.Name (include "mychart.name" .)
   | trunc 63 | trimSuffix "-" -}}
{{- end -}}
{{- end -}}
```

4 COMMON HELPER FUNCTIONS

FUNCTION	WHAT IT DOES	EXAMPLE
define	Creates a reusable named template	{{ define "my.name" }}...{{ end }}
include	Includes a named template and returns the result	{{ include "my.name" . }}
template	Similar to include, but prints directly	{{ template "my.name" . }}
nindent	Adds a newline and indents the text	{{ include "my.labels" . | nindent 4 }}
default	Sets a default value if value is empty	{{ .Values.replicaCount | default 1 }}
quote	Wraps a value in double quotes	{{ .Values.image | quote }}

5 USING HELPERS IN TEMPLATES

Reuse names and labels everywhere.
Example in deployment.yaml

```
metadata:
  name:  {{ include "mychart.fullname" . }}
  labels:
    {{ include "mychart.labels" . | nindent 4 }}
---
---
{{/* labels helper */}}
{{- define "mychart.labels" -}}
app.kubernetes.io/name: {{ include "mychart.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
app.kubernetes.io/version: {{ .Chart.AppVersion }}
{{- end -}}
```

6 WHY REUSE TEMPLATE LOGIC?

- Avoid duplicated YAML.
- Change logic in one place.
- Keep charts consistent.
- Reduce mistakes.
- Make charts easier to read and review.
- Improve scalability as charts grow.

7 AVOID DUPLICATED YAML

DON'T DO THIS

```
metadata:
  labels:
    app.kubernetes.io/name: myapp
    app.kubernetes.io/instance:
      myrelease
    app.kubernetes.io/managed-by: Helm
    app.kubernetes.io/version: 1.0.0
```

• Hard to maintain & easy to break.

DO THIS INSTEAD

```
metadata:
  labels:
    {{ include "mychart.labels" . | nindent 4 }}
```

Easy to maintain & update.

8 KEEP CHARTS MAINTAINABLE


Organize templates clearly.


Use helpers for names and labels.


Follow naming standards.


Validate before you deploy.


Review changes carefully.


Keep it simple, not clever.

CONFIGMAPS, SECRETS, AND APPLICATION CONFIGURATION

MANAGE CONFIGURATION THE RIGHT WAY

1 TEMPLATING CONFIGMAPS

Store non-sensitive configuration.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "mychart.fullname" . }}-config
  labels:
    {{- include "mychart.labels" . | nindent 4 }}
data:
  app.properties: |
    app.name={{ .Values.app.name }}
    log.level={{ .Values.log.level }}
    feature.enabled={{ .Values.feature.enabled }}
```

2 Ttemplating Secret References

Store sensitive values outside the chart (e.g., via Secret).

```
apiVersion: v1
kind: Secret
metadata:
  name: {{ include "mychart.fullname" . }}-secret
type: Opaque
stringData:
  username: {{ .Values.db.username | quote }}
  password: {{ .Values.db.password | quote }}
```

NEVER DO THIS

- Don't store real secrets in values.yaml.
- Don't commit secrets to Git.
- Don't expose secrets in templates or logs.
- Don't hardcode credentials.

Use external secret management.

3 ENVIRONMENT VARIABLES

Use ConfigMap and Secret values as env vars.

```
env:
 - name: APP_NAME
   valueFrom:
     configMapKeyRef:
       name: {{ include "mychart.fullname" . }}-config
       key: app.properties
 - name: DB_USERNAME
   valueFrom:
     secretKeyRef:
       name: {{ include "mychart.fullname" . }}-secret
       key: username
```

4 VOLUME-MOUNTED CONFIGURATION

Mount files into the container.

```
volumes:
  - name: app-config
    configMap:
      name: {{ include "mychart.fullname" . }}-config
volumeMounts:
  - name: app-config
    mountPath: /etc/myapp
    readOnly: true
```

TIP

Best for large config files, certificates, rules, or application configs.

5 WHY CHANGING A CONFIGMAP MAY NOT RESTART PODS

- ConfigMaps are mounted at runtime.
- Kubernetes does not restart Pods when a ConfigMap changes.
- Your app may not reload the new config automatically.

RESULT

The Pod keeps running with old configuration until it is restarted.

6 CHECKSUM ANNOTATIONS (AUTO RESTART)

Update annotation when ConfigMap changes to trigger a rolling restart.

```
metadata:
  annotations:
    checksum/config: {{ include (print $.Template.BasePath
      "/configmap.yaml") . | sha256sum }}
spec:
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print $.Template.BasePath
          "/configmap.yaml") . | sha256sum }}
```

TIP

When the ConfigMap changes, the checksum changes, so the Pod template changes and Kubernetes rolls out new Pods.

7 WHY SECRETS SHOULD NOT SIMPLY LIVE IN GIT

- Git is not a secret store.
- Anyone with repo access can read secrets.
- Secrets may leak through commit history, PRs, backups, or logs.
- Compliance and security risk!

8 EXTERNAL SECRET-MANAGEMENT PATTERNS

HashiCorp Vault · AWS Secrets Manager · Azure Key Vault · Google Secret Manager · External Secrets Operator · Sealed Secrets / SOPS

9 SECURITY REMINDER

- Use least privilege.
- Store secrets outside Git.
- Encrypt secrets at rest.
- Limit who can access secrets.
- Audit secret access.
- Rotate credentials regularly.
- Avoid exposing secrets in env vars when not necessary.
- Never log secrets.
- Review chart dependencies.
- Follow security best practices in every environment.

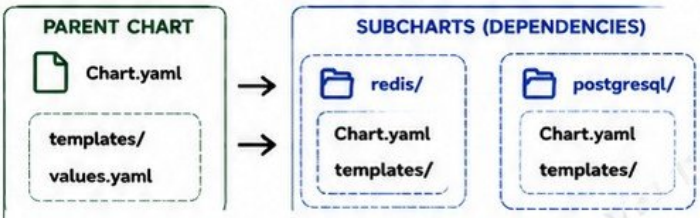
HELM DEPENDENCIES AND SUBCHARTS

Helm lets charts include other charts. This helps you reuse, organize, and maintain complex applications.

1 KEY CONCEPTS

- **PARENT CHART:** The main chart that declares dependencies.
- **SUBCHART:** A chart included inside the parent chart.
- The parent chart manages the release.
- Subcharts are deployed as part of the same release.

PARENT AND SUBCHART RELATIONSHIP



Subcharts live in the parent chart's charts/ directory after dependency update or build.

2 Chart.yaml DEPENDENCIES

Declare dependencies in the dependencies section.

```yaml
# Chart.yaml
apiVersion: v2
name: myapp
version: 1.0.0
dependencies:
- name: redis
  version: 18.1.0
  repository: https://charts.bitnami.com/bitnami
  alias: cache
- name: postgresql
  version: 15.5.20
  repository: https://charts.bitnami.com/bitnami
```

TIP: Use alias to rename a dependency when you include it multiple times.

3 MANAGING DEPENDENCIES

helm dependency update
Downloads the latest charts and updates the charts/ directory and Chart.lock.

helm dependency build
Builds the chart from the lock file.
It is offline-friendly and reproducible.

helm dependency list
Lists all dependencies.

Chart.lock (LOCK FILE)

- Generated automatically.
- Locks exact versions of dependencies.
- Ensures the same versions are used by everyone and in CI/CD.

4 PASSING VALUES TO DEPENDENCIES

Specific Subchart Values

```yaml
# values.yaml
redis:
  auth:
    enabled: true
  master:
    persistence:
      size: 5Gi
```

Values go under the subchart name (redis, postgresql).

Global Values

```yaml
# values.yaml
global:
  imageRegistry: registry.example.com
  storageClass: gp3
  commonLabels:
    team: platform
    env: production
```

Global values are accessible by all subcharts using .Values.global.*

5 WHEN TO USE DEPENDENCIES

USE DEPENDENCIES WHEN:

- The dependency is tightly coupled.
- You want a simple, single release.
- The dependency is an implementation detail (e.g., Redis, Postgres, Nginx).
- You want version alignment.
- You need default configuration that works out of the box.

USE INDEPENDENT RELEASES WHEN:

- The dependency is shared by many applications.
- Different teams manage the dependency.
- You need independent lifecycle and upgrade control.
- You need separate scaling, security, or backup policies.

6 EXAMPLE: HOW HELM HANDLES DEPENDENCIES

JUNIOR ENGINEER TIP:

Dependencies make charts easier to reuse, but overusing them can create hidden complexity. Always understand what your chart installs and how it will be managed in production.

UPGRADING APPLICATIONS SAFELY

Helm upgrades let you change your application version or configuration and apply the changes to a running release.

1 HELM UPGRADE COMMANDS

UPGRADE AN EXISTING RELEASE

```
>_ helm upgrade myapp ./mychart -f values.yaml
```

UPGRADE OR INSTALL IF NOT PRESENT

```
>_ helm upgrade --install myapp ./mychart -f values.yaml
```

--install makes the command idempotent. Great for CI/CD pipelines.

2 WHAT CAN YOU CHANGE?

CHANGE IMAGE TAGS

v1.0.0 → v1.1.0

```
>_ helm upgrade myapp ./mychart
   --set image.tag=v1.1.0
```

CHANGE CONFIGURATION

```
>_ helm upgrade myapp ./mychart
   -f values-prod.yaml
```

Helm applies only the changes. Unchanged resources are left untouched (strategic merge).

3 IMPORTANT UPGRADE FLAGS

	--atomic	If the upgrade fails, Helm automatically rolls back to the last successful release.
	--wait	Helm waits until all resources are in a ready state before marking the upgrade as successful.
	--timeout 5m	Sets the maximum time to wait for the upgrade to complete.
	--dry-run	Renders and validates the changes locally without applying them.

EXAMPLE: SAFE UPGRADE

```
>_ helm upgrade myapp ./mychart \
   --set image.tag=v1.1.0 \
   --atomic --wait --timeout 5m
```

TIP

Always use --atomic and --wait in production. They save you from broken deployments.

4 RELEASE REVISIONS

- Every upgrade creates a new revision.
- Helm keeps the full history.
- You can view and roll back to any revision.

```
>_ helm history myapp
```

REVISION	UPDATED	STATUS	CHART	APP VERSION	DESCRIPTION
3	2024-05-26 10:15:00	deployed	mychart-1.2.0	1.1.0	Upgrade complete
2	2024-05-25 09:40:00	superseded	mychart-1.1.0	1.0.0	Upgrade complete
1	2024-05-24 11:10:00	superseded	mychart-1.0.0	1.0.0	Install complete

5 DEPLOYMENT VERIFICATION

- A successful Helm command means Helm was able to send the changes to Kubernetes.
- It does NOT mean your application is healthy.
- Always verify after every upgrade.

```
>_ helm status myapp
>_ kubectl get pods -n myapp-ns
>_ kubectl get svc -n myapp-ns
>_ kubectl describe deploy myapp -n myapp-ns
>_ kubectl logs -n myapp-ns deploy/myapp
```

Pods Ready
Services Working
No Errors in Events

If not healthy, investigate and roll back if needed.

6 WHY A SUCCESSFUL HELM COMMAND DOES NOT AUTOMATICALLY MEAN A HEALTHY APPLICATION

Helm only ensures Kubernetes accepted the change.

Your application may crash after starting.

Readiness or liveness probes may fail.

Database or external services may be unreachable.

Misconfiguration may prevent the app from working correctly.

Performance issues will not be detected by Helm.

BEST PRACTICE

Upgrade in stages → Verify → Monitor → Promote.
Have a rollback plan → Always keep backups.

REMEMBER

Automation increases speed, but observation guarantees reliability.

HELM ROLLBACKS AND RELEASE RECOVERY

Rollbacks help you recover from failed or bad upgrades.
Use history, compare revisions, and roll back safely.

1 HELM HISTORY

Shows all revisions of a release.

```
>_ helm history myapp
```

REVISION	UPDATED	STATUS	CHART	APP VERSION	DESCRIPTION
5	2024-05-28 10:12:00	deployed	myapp-1.2.0	1.2.0	Upgrade complete
4	2024-05-28 09:50:00	failed	myapp-1.2.0	1.2.0	Upgrade failed
3	2024-05-27 16:30:00	deployed	myapp-1.1.0	1.1.0	Upgrade complete
2	2024-05-26 14:20:00	deployed	myapp-1.0.0	1.0.0	Install complete

2 RELEASE REVISIONS

- Every install or upgrade creates a new revision.
- Revisions are numbered (1, 2, 3, ...).
- Helm stores the full release history in Kubernetes Secrets.
- You can roll back to any previous revision.

Tip: Higher number = more recent revision.

3 HELM ROLLBACK

Roll back to a previous revision.

```
>_ helm rollback myapp 3
```

- Rolls back to revision 3.
- Creates a new revision (e.g., 6) with the previous successful state.
- Keeps history intact.

4 FAILED UPGRADES

- Upgrade may fail due to bad config, bad image, failing probes, or resource issues.
- Use history to find the last good revision.
- Use rollback to restore a stable state.

Example:
Revision 5 failed.
Rollback to revision 3 (last known good).

5 WHAT ROLLBACK CHANGES

- Restores the Kubernetes manifests and values from the target revision.
- Recreates or updates resources to match that revision.
- Updates the release status and history.
- Does NOT delete old Kubernetes objects that are not managed by the chart.

6 KUBERNETES STATE AFTER ROLLBACK

- Resources managed by the chart are rolled back.
- Unmanaged resources remain unchanged.
- Pods are restarted with the previous spec.
- Services, ConfigMaps, Secrets, and other objects are reverted to the target revision.
- External systems (DBs, queues, storage) are NOT rolled back automatically.

7 DATABASE MIGRATION COMPLICATIONS

- Rollback does not roll back your database.
- Schema changes (migrations) usually cannot be undone.
- Rolling back the app may be incompatible with the new schema.
- Always design migrations to be backward compatible when possible.
- Use versioned migrations and test downgrade scenarios.

PRO TIP: Treat database migrations as a separate concern. Have a plan for forward-only or reversible changes.

8 VERIFICATION AFTER ROLLBACK

After rollback, always verify:

- helm status myapp
- kubectl get all -n <namespace>
- kubectl get pods -n <namespace>
- kubectl describe deploy myapp -n <namespace>
- Check application health (liveness/readiness)
- Check logs for errors
- Verify external dependencies

9 PRODUCTION AWARENESS: ROLLBACK IS A STRATEGY, NOT A GUARANTEE

- Rollback returns your manifests, not your entire system to a previous state.
- Data changes are permanent unless you have backups and restore plans.
- External systems (DB, queues, APIs, storage) may not be reversible.
- Rollback does not fix the root cause. Investigate before the next upgrade.
- Test rollbacks in staging. Practice before you need it in production.

KEY TAKEAWAY:
Use history to understand what happened. Roll back to a known good state.
Verify everything after rollback and fix the root cause before moving forward.

HOOKS, JOBS, AND DEPLOYMENT LIFECYCLE

Helm hooks let you run Kubernetes resources at specific points in the release lifecycle.

1 WHAT HELM HOOKS ARE

- Hooks are Kubernetes resources with special annotations that tell Helm when to run them.
- They are used for tasks like database migrations, notifications, backups, validations, and cleanup.
- Hooks are not part of the main release unless you design them that way.

2 HOOK LIFECYCLE EVENTS

PRE-INSTALL	POST-INSTALL	PRE-UPGRADE	POST-UPGRADE	PRE-DELETE
Runs before installing the release.	Runs after the release is installed.	Runs before upgrading the release.	Runs after the release is upgraded.	Runs before deleting the release.

3 HOOK ANNOTATIONS

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "-5"
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  template:
    spec:
      containers:
      - name: migrate
        image: myapp/migrate:latest
        command: ["/bin/sh","-c","./migrate.sh"]
      restartPolicy: Never
```

TIP: You can list multiple events separated by commas.
Example: pre-install,pre-upgrade

4 HOOK WEIGHTS

- Hooks are executed in order of weight.
- Lower weight runs first.
- Default weight is `0`.
- Use negative numbers to run something earlier.

-10	→	-5	→	0	→	5
Runs first		Runs next		Default		Runs last

5 HOOK DELETION POLICIES

- `before-hook-creation` : Delete previous hook before creating a new one.
- `hook-succeeded` : Delete hook after success.
- `hook-failed` : Delete hook after failure.
- `before-hook-creation,hook-succeeded` : Common for Jobs.
- `hook-succeeded,hook-failed` : Clean up no matter the result.

6 DATABASE MIGRATION JOB EXAMPLE

- Use a pre-upgrade hook Job to run database migrations.
- Ensure it completes successfully before the app updates.
- If the migration fails, the upgrade should stop (use --atomic).
- Make the Job idempotent (safe to run multiple times).
- Never run destructive migration scripts in hooks.

```
>_ helm upgrade myapp ./mychart ---atomic --wait --timeout 10m
```

7 WHY BADLY DESIGNED HOOKS CAN BLOCK PRODUCTION RELEASES

- Hook Job stuck in Pending (no resources available).
- Hook container image is missing or broken.
- Hook script hangs or waits for user input.
- Hook depends on external service that is down.
- Bad hook weight causes deadlock or wrong order.
- Hook never completes → Helm waits forever.

8 HELM DEPLOYMENT LIFECYCLE WITH HOOKS

PRE-INSTALL	INSTALL / UPGRADE	POST-INSTALL / POST-UPGRADE	APPLICATION RUNNING	PRE-DELETE	DELETE RELEASE
Run hooks	Deploy or upgrade resources	Run hooks	Release is active	Run hooks	Release removed

9 PRODUCTION AWARENESS

Hooks are powerful but risky. Test them in non-production environments first. Make them fast, reliable, and idempotent. Rollback may not undo external changes made by hooks (like database migrations). Always have a recovery plan.

VERIQTA

CHART SECURITY AND PRODUCTION HARDENING

Secure by design. Protect your workloads, data, and cluster.

1 LEAST PRIVILEGE

- Grant only the permissions that are required.
- Do not use cluster-admin in Helm charts.
- Reduce the blast radius of a compromise.

Principle: Minimum access to do the job.

2 KUBERNETES RBAC

- Create Roles and RoleBindings for specific tasks.
- Scope permissions by namespace whenever possible.
- Avoid wildcards in rules.

3 SERVICEACCOUNTS

- Use dedicated ServiceAccounts per application.
- Do not use the default ServiceAccount.
- Automount tokens only when needed.

4 SECURITY CONTEXTS

- Run containers as non-root (runAsNonRoot: true).
- Drop unnecessary capabilities.
- Set readOnlyRootFilesystem: true.
- Use seccomp, AppArmor or SELinux profiles.

5 RESOURCE REQUESTS AND LIMITS

- Set CPU and memory requests and limits.
- Prevent noisy neighbor issues.
- Protect cluster stability and ensure fair usage.

```
resources:
  requests:
      cpu: 100m
      memory: 128Mi
  limits:                    memory:
       cpu: 500m              memory:512Mi
```

6 AVOIDING PRIVILEGED CONTAINERS

- Do NOT run privileged containers.
- Avoid hostPID, hostIPC, and hostNetwork unless absolutely necessary.
- Drop dangerous Linux capabilities.

7 IMAGE TAGS AND IMMUTABLE VERSIONS

- Always use immutable tags (digest or version).
- Avoid :latest in production.
- Example: myapp@sha256:abcd... or myapp:1.2.3

8 SECRETS EXPOSURE RISKS

- Never store secrets in values.yaml or templates.
- Do not expose secrets in env vars, logs, or command args.
- Use Kubernetes Secrets or external secret managers.
- Encrypt secrets at rest and in transit.

9 CHART PROVENANCE AND TRUSTED SOURCES

- Verify chart provenance using signed charts.
- Use trusted Helm repositories only.
- Enable provenance verification in CI/CD.
- Example:

```
helm verify mychart-1.0.0.tgz
```

12 DEPENDENCY REVIEW

- Review all chart dependencies.
- Check for active maintenance.
- Verify versions and security advisories.
- Avoid unnecessary dependencies.
- Keep Chart.lock committed.

11 SECURITY SCANNING IN CI/CD

- Scan container images for vulnerabilities.
- Scan Helm charts for misconfigurations.
- Scan IaC and templates.
- Fail the pipeline on high/critical issues.

Tools Examples:
- Trivy
- Grype
- Kube-score
- Polaris
- Conftest / OPA
- Falco

PRODUCTION HARDENING CHECKLIST

- ☑ Least privilege everywhere
- ☑ RBAC scoped to namespace
- ☑ Dedicated ServiceAccounts
- ☑ Security contexts enforced
- ☑ Resource requests and limits set
- ☑ No privileged containers
- ☑ Immutable image versions
- ☑ Secrets managed securely
- ☑ Charts verified and from trusted sources
- ☑ Dependencies reviewed and locked
- ☑ Security scanning in pipeline
- ☑ Continuous monitoring and auditing

REMEMBER: ROLLBACK CAN RESTORE RESOURCES, BUT NOT YOUR DATA.
SECURE CHARTS PREVENT PROBLEMS BEFORE THEY HAPPEN.

HELM IN CI/CD AND GITOPS

Automate, collaborate, and deploy applications to Kubernetes reliably with Helm.

1 THE CI/CD + HELM FLOW

1. Developer commits code
2. CI builds application
3. Container image pushed to registry
4. Helm chart validated
   - helm lint
   - helm template
   - security scan
5. Values files updated
   - dev
   - staging
   - prod
6. Deployment triggered
   Helm install or upgrade
7. Kubernetes cluster
   Application is updated

2 HELM IN POPULAR CI/CD TOOLS

GITHUB ACTIONS
Run helm lint, helm template, and helm upgrade using workflow YAML.

GITLAB CI
Use .gitlab-ci.yml to validate charts and deploy with Helm.

JENKINS
Pipeline jobs run Helm commands to deploy to Kubernetes.

3 HELM WITH GITOPS TOOLS

ARGO CD
Argo CD watches your Git repo. When Helm charts or values change, it automatically syncs the desired state to the cluster.

FLUX
Flux continuously reconciles your Git repo with the cluster using Helm charts.

4 IMPERATIVE HELM vs DECLARATIVE GITOPS

IMPERATIVE HELM (MANUAL / CLI DRIVEN)

- You run helm install / upgrade commands.
- State is stored on the cluster.
- Changes may drift from Git.
- Good for quick tasks and admin operations.
- Example: helm upgrade myapp ./chart -f values.yaml

DECLARATIVE GITOPS (GIT DRIVEN)

- Desired state is defined in Git.
- Git is the single source of truth.
- Tools like Argo CD or Flux apply changes.
- Drift is detected and corrected automatically.
- Better for production and team collaboration.

5 PROMOTING BETWEEN ENVIRONMENTS

DEVELOPMENT
Auto deploy on every commit. Fast feedback.

→

STAGING
Test with production-like values and data.

→

PRODUCTION
Promote tested version safely.

PROMOTION STRATEGIES

- Update image tag in values file
- Commit change to Git
- GitOps tool syncs to target env
- Verify and monitor rollout

6 EXAMPLE REPO STRUCTURE (GITOPS FRIENDLY)

```
repo/
├── charts/
│   └── myapp/
└── values/
    ├── dev-values.yaml
    ├── staging-values.yaml
    └── prod-values.yaml
└── environments/
    ├── dev/
    │   └── kustomization.yaml
    ├── staging/
    │   └── kustomization.yaml
    └── prod/
        └── kustomization.yaml
```

charts/
Helm charts live here.

values/
Environment-specific values.

environments/
GitOps tool reads manifests here (Argo CD / Flux).

JUNIOR ENGINEER TIPS

- Always validate charts before deploying.
- Keep values files small and focused.
- Use Git for every change.
- Promote the same artifact, not new builds.
- Monitor deployments after every release.

TROUBLESHOOTING FAILED HELM DEPLOYMENTS

A PRACTICAL GUIDE TO FIND, DIAGNOSE, AND FIX HELM ISSUES FAST

1 ESSENTIAL HELM COMMANDS

`helm status <release>`	Show release status, last chart, namespace and basic info.
`helm history <release>`	Show revision history and status of each release.
`helm get manifest <release>`	Show the complete rendered manifest of the release.
`helm get values <release>`	Show the values used (merged values).
`helm template <release> <chart>`	Render local templates without installing.

2 KUBERNETES DEBUG COMMANDS

`kubectl get all -n <ns>`	List all resources in the namespace.
`kubectl describe <resource> <name> -n <ns>`	Show detailed information about a resource.
`kubectl logs <pod> -n <ns>`	View logs from a Pod or container.
`kubectl get events -n <ns>`	Show recent events in the namespace.

3 COMMON HELM FAILURE CATEGORIES

Template rendering errors	Invalid template syntax, missing values, wrong indentation.
Invalid values	Wrong data type, missing required value, invalid format.
ImagePullBackOff	Image not found, wrong tag, or registry authentication error.
CrashLoopBackOff	Application starts but crashes repeatedly.
Failed probes	Liveness or readiness probe failing.
Pending Pods	Insufficient resources, scheduling issues, or pending dependencies.
Failed hooks	Pre/post hooks failed, blocking the release.

4 TROUBLESHOOTING FLOW

HELM — Check release status and history

RENDERED MANIFEST — Inspect rendered YAML for errors

K8S RESOURCE — Check resource status and events

POD — Check pod logs and conditions

APPLICATION — Verify app health and functionality

KEY IDEA: Move step by step from Helm to the application. Find the first failing point, fix the root cause, then re-verify.

5 PRACTICAL TROUBLESHOOTING WORKFLOW (DETAILED)

1	Check Release Status	`helm status <release>`	Look for FAILED, pending-upgrade, or warnings.
2	Check History	`helm history <release>`	Identify when the failure started.
3	Inspect Rendered Manifest	`helm get manifest <release>`	Verify resources, values, and configurations.
4	Check Values	`helm get values <release>`	Confirm the values being used are correct.
5	Render Locally	`helm template <release> <chart>`	Catch template or values issues before deploy.
6	Check Kubernetes Resources	`kubectl get all -n <ns>`	See what was created and their status.
7	Describe Problem Resource	`kubectl describe <resource> -n <ns>`	Read events, errors, and conditions.
8	Check Pod Logs	`kubectl logs <pod> -n <ns>`	Find application errors and startup failures.
9	Verify Probes and Health	`kubectl describe pod <pod> -n <ns>`	Check liveness/readiness probe results.
10	Fix Root Cause and Re-Deploy	`helm upgrade --install <release> <chart> -f values.yaml`	Validate the fix and monitor the deployment.

JUNIOR ENGINEER TIP

Read the error message carefully. It usually tells you exactly where the problem is. Don't guess—observe, verify, then fix.

PRODUCTION REMINDER

Always test changes in a safe environment first. Use --atomic and --wait during upgrades.

PRODUCTION CHART DESIGN AND RELEASE STRATEGY

BUILD RELIABLE HELM CHARTS AND DELIVER SAFE, REPEATABLE RELEASES

1 ENVIRONMENTS: DEV → STAGING → PROD

DEVELOPMENT

- Fast feedback
- Developer friendly
- Lower resources
- More frequent changes

→

STAGING

- Production-like
- Testing & validation
- Performance testing
- Pre-production verification

→

PRODUCTION

- Highly available
- Secure & stable
- Change controlled
- Monitor & audit

Promote the same chart with different values.

2 BASE VALUES vs ENVIRONMENT OVERRIDES

values.yaml (base values)

+

values-<env>.yaml (environment overrides)

Example Structure

```
charts/
└ myapp/
   ├ values.yaml
   ├ values-dev.yaml
   ├ values-staging.yaml
   └ values-prod.yaml
```

Keep base values generic. Override only what differs per environment.

3 SEMANTIC CHART VERSIONS

Follow Semantic Versioning for your charts.

- MAJOR: Breaking changes
- MINOR: New features
- PATCH: Bug fixes

Example: 1.4.2

4 APPLICATION VERSIONS

Your chart deploys an application version (image tag).

Keep chart version and app version independent.

Example: app: 2.3.1

5 OCI REGISTRIES & PUBLISHING

- Store charts in OCI registries (Harbor, GHCR, ECR, ACR, GCR).
- Versioned, secure, and access controlled.
- `helm push oci://<registry>/<repo>`

6 RELEASE OWNERSHIP

Every release should have:

- Owner / team
- Contact
- Purpose
- Runbook link
- Change history

7 VERSION PINNING

Always pin chart dependencies and app images.

- Chart dependencies in Chart.lock
- Image: use specific tags (avoid :latest)
- Ensures reproducible deployments

8 CHANGE REVIEW

- Peer review for all chart and values changes.
- Validate before merging.
- Use pull requests.
- Require approvals.

DEPLOYMENT WINDOWS

- Define when changes can be deployed.
- Avoid peak traffic times.
- Communicate window and impact.
- Use --atomic and --wait for safer releases.

ROLLBACK PLANNING

- Know how to roll back before you deploy.
- Test rollback in staging.
- Keep previous working version available.
- Automate where possible.

AVOID ENVIRONMENT CONFIG DRIFT

- Store all environment values in Git.
- Promote through Git, not manual changes.
- Use the same chart for all environments.
- Keep environments aligned.

DESIGN CHARTS TEAMS CAN MAINTAIN

- Simple, consistent, and well documented.
- Sensible defaults.
- Reusable helpers and patterns.
- Clear README and examples.
- Test templates and values changes.

JUNIOR ENGINEER TIP

A good Helm chart is not just about deploying resources.
It is about delivering reliability, consistency, and safety across environments.
Design for operations, not just for installation.

FINAL PRODUCTION LAB, SHIP AND RECOVER AN APPLICATION

YOUR MISSION

Build, deploy, break, and recover a real application using Helm in Kubernetes just like in production.

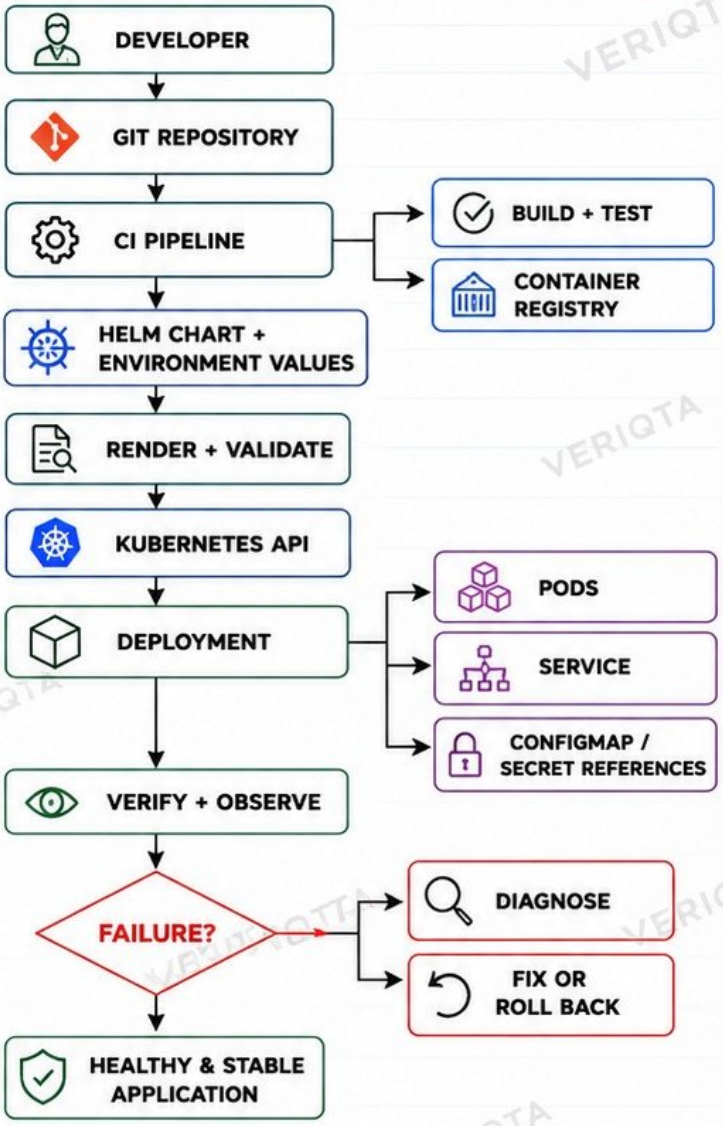
LAB STEPS

1	Build a production-ready Helm chart
2	Configure Deployment and Service
3	Add ConfigMap configuration
4	Reference secrets safely
5	Configure probes (liveness, readiness)
6	Set CPU and memory requests/limits
7	Render and validate manifests
8	Install into a namespace
9	Verify Pods and Services
10	Perform an application upgrade
11	Introduce a deliberately broken image tag
12	Diagnose the failed rollout
13	Inspect Helm and Kubernetes state
14	Roll back to the healthy revision
15	Verify recovery
16	Review release history

JUNIOR ENGINEER TIP

In production, things will fail. Great engineers ship with confidence and recover with speed.

FINAL ARCHITECTURE & WORKFLOW



KEY COMMANDS YOU WILL USE

```
helm create myapp
helm lint
helm template myapp
helm install myapp ./myapp \
    --namespace lab --create-namespace
helm status myapp -n lab
```

```
helm upgrademyapp ./myapp \
    --set image.tag=2.0.0
helm history myapp -n lab
helm rollback myapp 1 -n lab
helm get manifest myapp -n lab
```

```
helm get values myapp -n lab
kubectl get all -n lab
kubectl describe pod -n lab
kubectl logs -n lab <pod-name>
```

REMEMBER

A successful engineer not only ships applications, but also knows how to recover them.